

ESP32 Detection and Health Check Guide on Ubuntu 22.04

Overview

When working with ESP32 development boards, it is common to encounter situations where:

- The board is not detected by the computer.
- Code upload fails.
- The board appears to be dead.
- No serial output is visible.
- The board stops working after uploading a program.

Before assuming that the ESP32 hardware is damaged, perform the following checks. In many cases, the problem is related to the USB cable, drivers, permissions, baud rate, or firmware rather than the ESP32 itself.

Step 1: Verify USB Detection

Connect the ESP32 to the computer and run:

```
lsusb
```

Expected output examples:

```
Silicon Labs CP210x USB to UART Bridge
```

or

```
QinHeng Electronics CH340 USB-Serial Adapter
```

or

```
FTDI USB Serial Device
```

Result

If the device appears in the list:

- USB connection is working.
- USB-UART converter is detected by Linux.

If no device appears:

- Check the USB cable.
 - Try another USB port.
 - Try another computer.
-

Step 2: Check Serial Port Creation

Run:

```
ls /dev/ttyUSB*
```

or

```
ls /dev/ttyACM*
```

Example:

```
/dev/ttyUSB0
```

Result

If a serial device appears:

- Linux has successfully loaded the driver.
- Communication with the USB-UART chip is possible.

If no device appears:

- USB-UART driver issue.
 - Faulty cable.
 - Faulty USB-UART chip.
-

Step 3: Monitor Kernel Messages

Run:

```
dmesg -w
```

Then reconnect the ESP32.

Expected output:

```
cp210x converter now attached to ttyUSB0
```

or

```
ch341-uart converter now attached to ttyUSB0
```

Result

This confirms Linux has recognized the device correctly.

Step 4: Open the Serial Monitor

Install Screen:

```
sudo apt update  
sudo apt install screen
```

Open the serial port:

```
screen /dev/ttyUSB0 115200
```

Press the ESP32 RESET (EN) button.

Expected output:

```
rst:0x1 (POWERON_RESET)
ESP-ROM:esp32
```

or similar boot messages.

Result

If boot messages appear:

- ESP32 CPU is running.
- Bootloader is working.
- Flash memory is readable.
- UART communication is functioning.

This is one of the quickest methods to verify that the ESP32 hardware is healthy.

Step 5: Upload a Simple Test Program

Example:

```
void setup()
{
    Serial.begin(115200);
}

void loop()
{
    Serial.println("ESP32 Working");
    delay(1000);
}
```

Open the serial monitor:

```
screen /dev/ttyUSB0 115200
```

Expected output:

```
ESP32 Working
ESP32 Working
ESP32 Working
```

Result

If messages appear:

- Program upload succeeded.
- Flash memory is functioning.
- CPU is executing code correctly.

Step 6: Hardware Blink Test

Connect an LED:

GPI02 ---- 220Ω ---- LED ---- GND

Program:

```
#define LED 2

void setup()
{
    pinMode(LED, OUTPUT);
}

void loop()
{
    digitalWrite(LED, HIGH);
    delay(1000);

    digitalWrite(LED, LOW);
    delay(1000);
}
```

Expected behavior:

LED ON for 1 second
LED OFF for 1 second

Result

If the LED blinks:

- GPIO subsystem is functioning.
- CPU execution is normal.
- The board is operational.

Common Problems and Solutions

Problem	Possible Cause	Solution
No device in lsusb	Bad cable	Replace cable
No /dev/ttyUSB0	Driver issue	Reconnect board or install drivers
Permission denied	User permissions	Add user to dialout group
Garbage characters	Wrong baud rate	Use correct baud rate
Upload fails	Boot mode issue	Hold BOOT button while uploading
No boot messages	Hardware fault or power issue	Check power supply

Fix Linux Permission Errors

Check current groups:

```
groups
```

If "dialout" is missing:

```
sudo usermod -aG dialout $USER
```

Logout and login again.

Recommended Diagnostic Commands

These commands should be the first thing to run whenever an ESP32 appears not to work:

```
lsusb  
ls /dev/ttyUSB*  
dmesg -w  
screen /dev/ttyUSB0 115200
```

If boot messages appear after pressing RESET, the ESP32 hardware is usually healthy and the issue is likely related to software, configuration, or firmware rather than a damaged microcontroller.

Conclusion

The most useful command for quickly determining whether an ESP32 is alive is:

```
screen /dev/ttyUSB0 115200
```

By opening the serial port and observing the boot messages after pressing RESET, developers can immediately verify the health of the ESP32 CPU, flash memory, UART communication, and bootloader without writing additional test code.

This simple technique can save significant debugging time and should be part of every embedded developer's troubleshooting workflow.